Trabajo Práctico 1 – Ejercicios extra

SISTEMAS OPERATIVOS

Tracks

1. Archivos y llamadas al sistema (3/10) **3**

2. Pipelines (7/10) **4**

3. Inspección del sistema (5/10) **5**

4. Entorno del shell (5/10) **6**

5. Procesos (3/10) **7**

1. Archivos y llamadas al sistema (3/10)

TRACK – 3/10

Descripción: cómo los programas acceden a archivos a través de llamadas al sistema. Se utilizan C, Python y código fuente de comandos de Linux.

1. Leer un archivo en C usando `open` y `read`.
2. Repetir el ejercicio en Python usando `open` y `read`.
3. Copiar un archivo en C usando `open`, `read` y `write`.
4. Implementar la misma copia en Python.
5. Encontrar el código fuente de `ls` y `cat`.
6. Identificar cómo abren, leen y muestran archivos.

2. Pipelines (7/10)

TRACK – 7/10

Descripción: cómo conectar comandos y transformar datos usando streams. Se utilizan pipes, redirecciones, `grep`, `sort`, `uniq`, `wc`, `xargs`, `sed`, `awk`, `curl`, `wget` y `jq`; también se exploran sus opciones mediante las páginas de manual.

1. Redirigir la salida con `>` y `>>`.
2. Conectar comandos usando `|`.
3. Filtrar resultados con `grep`.
4. Ordenar, eliminar duplicados y contar con `sort`, `uniq` y `wc`.
5. Ejecutar un comando por cada resultado usando `xargs`.
6. Editar texto en un stream usando `sed`.
7. Extraer campos y calcular valores usando `awk`.
8. Descargar datos de premios Nobel:

   ```
   curl -o prizes.json https://api.nobelprize.org/v1/prize.json
   ```

9. Procesar el JSON con `jq`:
   - contar premios por año y categoría;
   - listar premiados de un año;
   - contar nombres que comienzan con una letra;
   - encontrar premiados múltiples.

3. Inspección del sistema (5/10)

TRACK – 5/10

Descripción: cómo Linux expone el estado del sistema. Se utilizan `/proc`, `ps`, `top`, `free`, `df` y `du`; también se exploran sus opciones mediante las páginas de manual.

1. Inspeccionar información de CPU en `/proc/cpuinfo`.
2. Inspeccionar memoria en `/proc/meminfo`.
3. Inspeccionar uptime y load average.
4. Inspeccionar filesystems montados y uso de disco.
5. Explorar `/proc/<pid>` para procesos activos.
6. Usar `ps`, `top`, `free`, `df` y `du`.
7. Comparar la salida de estos comandos con la información de `/proc`.
8. Escribir un script que extraiga valores relevantes y muestre un reporte conciso. Este ejercicio conecta con el track de Pipelines.

4. Entorno del shell (5/10)

TRACK – 5/10

Descripción: cómo el shell configura y lanza programas. Se utilizan variables de entorno, `PATH`, exportación, herencia y `env`; también se exploran sus opciones mediante las páginas de manual.

1. Inspeccionar variables usando `env`, `printenv` y `echo`.
2. Guardar el valor actual de `PATH`.
3. Ejecutar `export PATH=""` y observar qué comandos fallan.
4. Ejecutar comandos usando rutas absolutas.
5. Restaurar `PATH`.
6. Definir una variable sin exportarla y observar su ausencia en un shell hijo.
7. Exportar la variable y repetir la prueba.
8. Modificar `HOME`, `PATH` y variables propias usando `env`.

5. Procesos (3/10)

TRACK – 3/10

Descripción: cómo Linux administra programas en ejecución. Se utilizan jobs, estados de procesos, señales, `ps`, `top` y `/proc`; también se exploran sus opciones mediante las páginas de manual.

1. Ejecutar comandos en foreground y background.
2. Usar `jobs`, `fg` y `bg`.
3. Inspeccionar procesos con `ps` y `top`.
4. Pausar y reanudar procesos.
5. Enviar señales usando `kill`.
6. Comparar terminación controlada y forzada.
7. Observar procesos padre e hijo.
8. Inspeccionar estados en `/proc/<pid>/status`.